

Group C – Topology

Angel Perez, Justin Emerick, Daniel Nethala, Mathew Goncalves

Saint Leo University

COM 430: Software Engineering

Professor Paquette

May 31, 2026

Table of Contents

1. Executive Summary	3
2. System Models and Topology	3
2.1 Context Diagram	3
2.2 UML Overview	4
2.3 Class Diagram	5
2.4 Sequence Diagram	6
2.5 Use Case Diagram	8
3. Infrastructure and Topology	9
4. Update Project Plan/Gantt Chart	10
5. Agile Estimation Session	11
5.1 Planning Poker Methodology	11
5.2 Agile Poker Estimation Results	12
5.3 Analysis	13
6. Meeting Notes	13
7. Conclusion	14

Executive summary

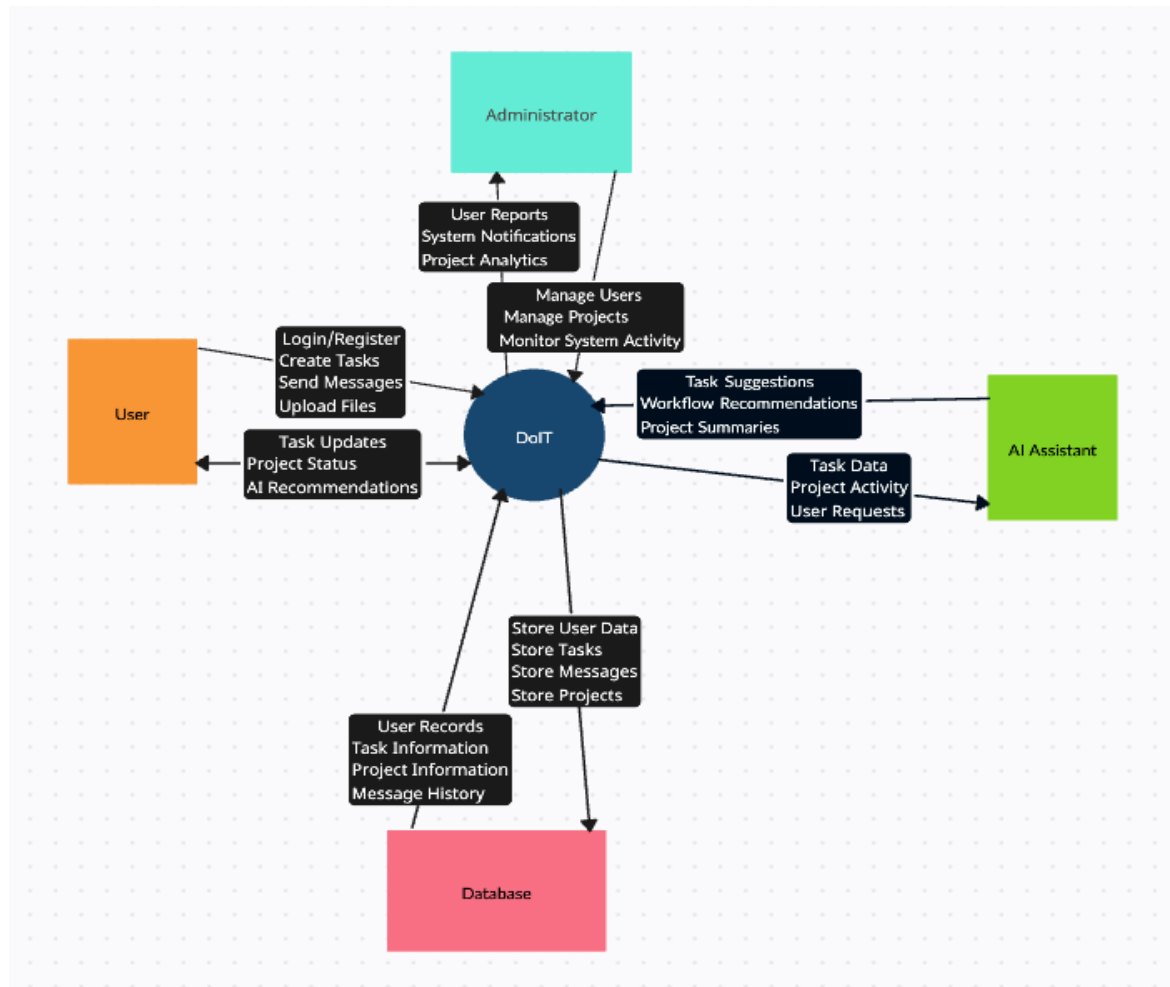
DoIT is a project management and collaboration application that is being developed to help teams organize projects, assign tasks, communicate, and track progress in one place. The goal of the application is to make collaboration easier by combining several tools into a single platform. The application will include features such as task management, team messaging, file sharing, project dashboards, and an AI assistant that can help users organize tasks and provide project summaries. The project is being developed using a DevOps approach, which includes Development, Testing, Staging, and Production environments. This report provides an update on the current progress of the project. It includes system models, infrastructure planning, project scheduling updates, agile estimation results, and meeting notes completed during this stage of development.

System Models and Topology

Context Diagram

The context diagram provides an overview of the DoIT system and its interactions with entities. It shows how users, administrators, the AI assistant, and the database communicate with the application. The purpose of this diagram is to show the flow of information between the system and its primary actors without focusing on internal system processes.

Figure 1. DoIT Context Diagram



Note. This diagram shows the primary entities that interact with the DoIT platform and the flow of information between those entities and the system.

UML Overview

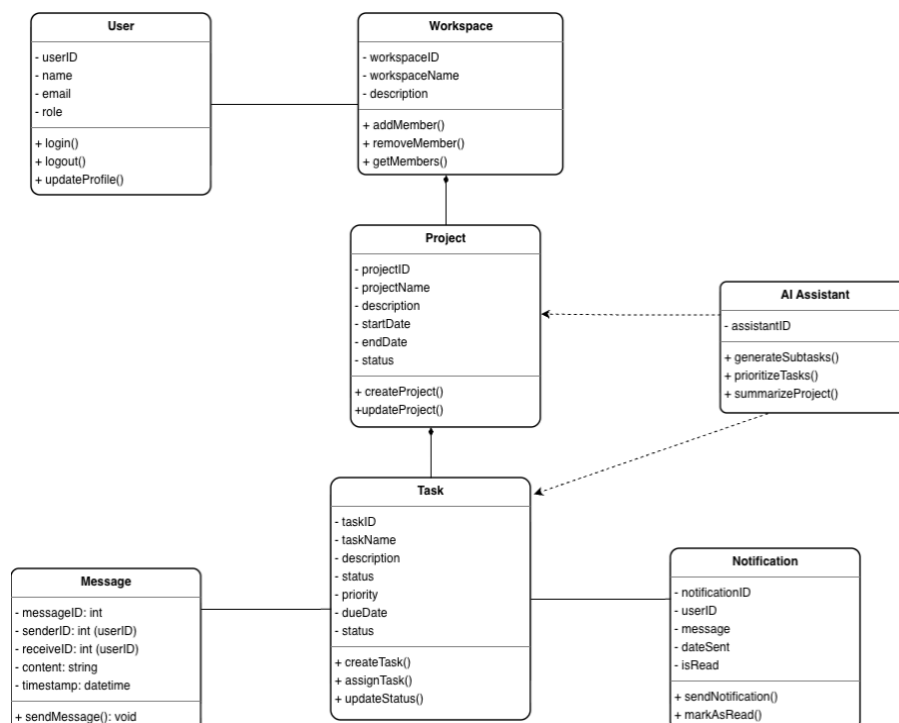
Unified Modeling Language (UML) diagrams were developed to help visualize the overall design and functionality of the DoIT platform. UML modeling allows the development team to represent system requirements, user interactions, and software components in a structured format before implementation begins. Creating these models helps ensure that all team members share a common understanding of how the application is intended to function and how different features relate to one another. The diagrams included in this section focus on the

primary users of the system, the services available to those users, and the relationships between the major components that make up the DoIT platform.

Class Diagram

The class diagram provides a structural representation of the DoIT platform by identifying the primary classes, their attributes, and the relationships that exist between them. While the use case diagram focuses on user interactions, the class diagram focuses on the underlying structure of the application itself. Developing this diagram allows the team to better understand how information is organized and exchanged throughout the system. The classes included represent key features such as users, workspaces, projects, tasks, notifications, messaging, and AI-assisted functionality that support collaboration and project management within the platform.

Figure 2. DoIT Class Diagram

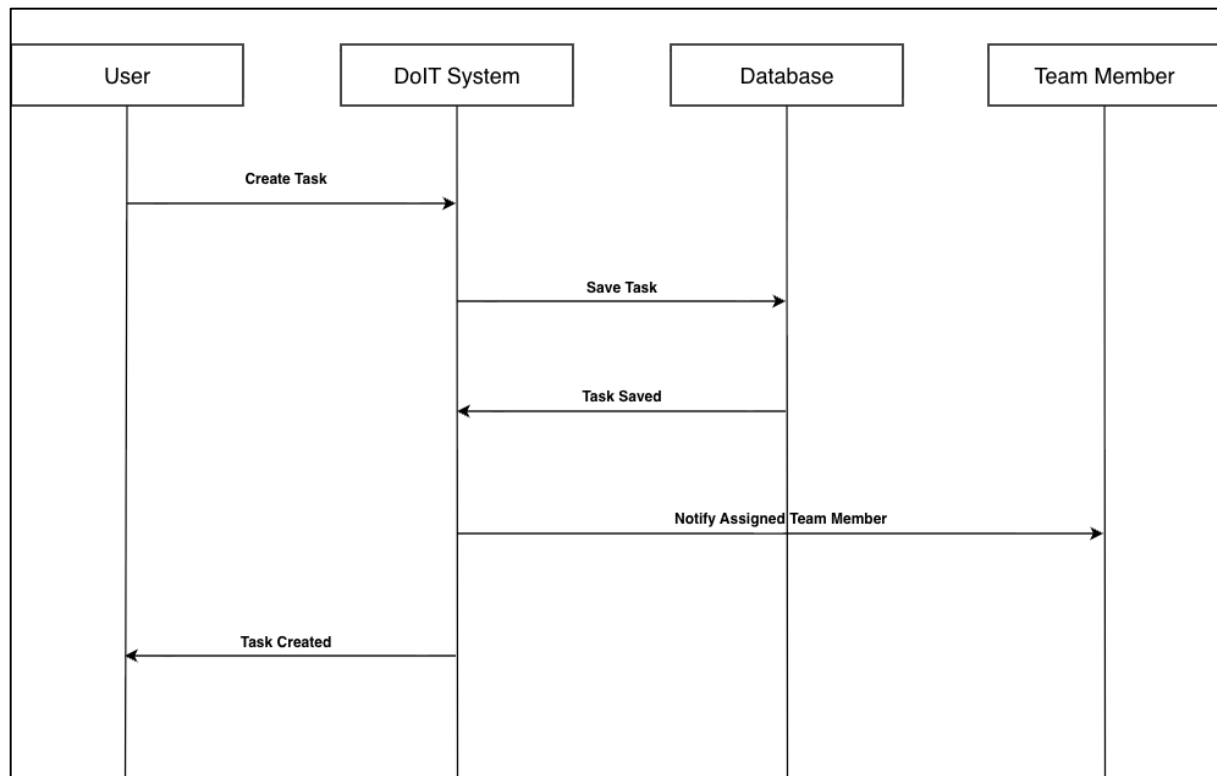


Note. The DoIT Class Diagram illustrates the major classes that support the platform, including users, projects, tasks, messaging, notifications, and AI-assisted functionality. The relationships between classes demonstrate how information is organized and exchanged throughout the system.

Sequence Diagram

The sequence diagram provides a visual representation of how information flows between users and system components during a specific activity within the DoIT platform. While the class diagram focuses on the structure of the system, the sequence diagram focuses on the order of interactions that occur when a process is performed. This model illustrates the communication that takes place when a task is created, including interactions between the project manager, the DoIT system, the database, and team members. Developing this diagram helps the team better understand how information moves throughout the application and how multiple system components work together to support project collaboration and task management.

Figure 3. DoIT Task Creation Sequence Diagram



Note. The DoIT Task Creation Sequence Diagram illustrates the sequence of interactions that occur when a project manager creates a task within the platform. The diagram demonstrates how the DoIT system processes the request, stores task information in the database, and notifies team members of newly assigned work.

Use Case Diagram

The use case diagram illustrates the primary functions of the DoIT application and the interactions between users, administrators, and the AI assistant. Users can perform actions such as creating projects, managing tasks, sending messages, and uploading files. Administrators are responsible for managing users and monitoring system activity. The AI assistant supports users by providing task recommendations, project summaries, and help prioritize tasks. This diagram

provides a visual representation of the system requirements and how each use case interacts with the platform.

Figure 4. DoIT Use Case Diagram



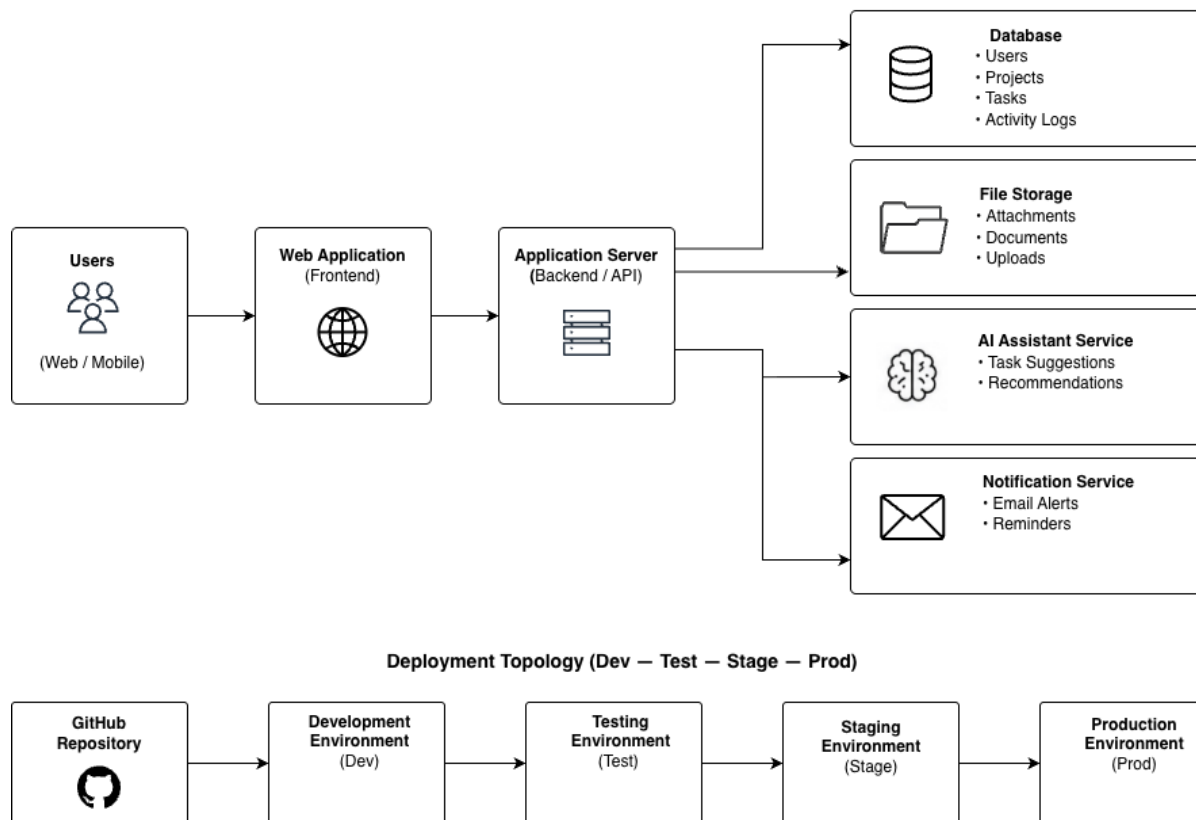
Note. This diagram identifies the use cases within the DoIT platform, including users, administrators, and the AI assistant, along with functions supported by the system.

Infrastructure and Topology

The infrastructure and topology design provides a conceptual view of the technical architecture supporting the DoIT platform. In addition to understanding how users interact with the system, it is important to understand how the underlying services and environments work together to support application functionality. This diagram illustrates the major infrastructure

components that support the platform, including the frontend application, backend services, database storage, AI-assisted services, notification systems, and deployment environments. The model also demonstrates how application updates progress through the DevOps lifecycle from development to testing, staging, and production environments before being released to end users.

Figure 5. DoIT Infrastructure and Topology Diagram

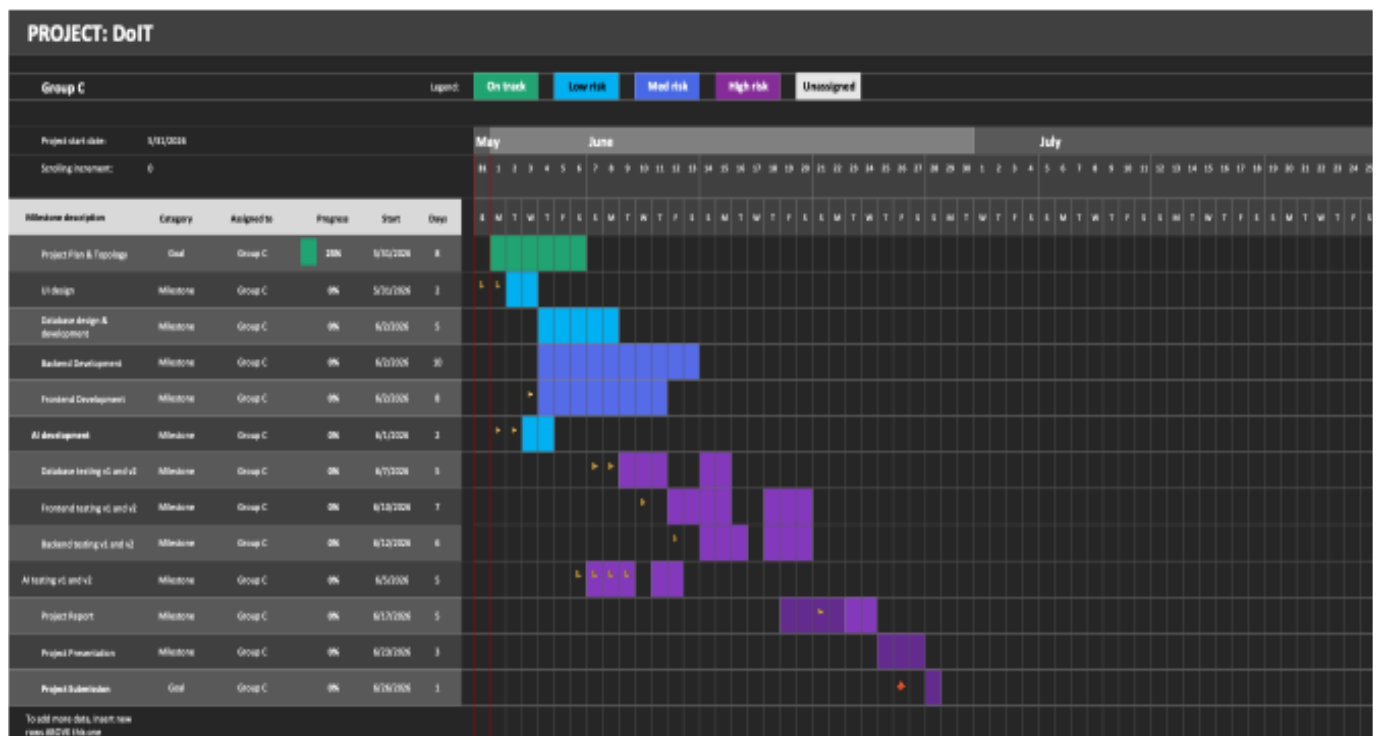


Note. The infrastructure and topology diagram illustrates the primary components of the DoIT platform, including the frontend application, backend services, supporting data services, and the DevOps deployment workflow from development through production environments.

Update Project Plan/Gantt Chart

Updates to the Gantt chart include updated progress of application planning and design. Also, the dates mentioned on the Gantt chart are now accurately reflected on the chart grid. The following updated Gantt chart will be constantly updated to reflect the current progress of application development.

Figure 6. Updated Project Schedule



Note. The updated project schedule reflects the current project timeline, development milestones, task dependencies, and planned progression through the DevOps lifecycle from development to production.

Agile Estimation Session

Planning Poker Methodology

Agile Planning Poker is an estimation technique used by software development teams to determine the relative effort required to complete project features and user stories. The method encourages collaboration among team members while reducing bias that can occur when estimates are discussed openly before voting. For the DoIT project, Planning Poker was used to estimate the complexity and development effort associated with the application's core functionality.

Participants

- Angel Perez
- Justin Emerick
- Daniel Nethala
- Mathew Goncalves

Estimation Method

The team utilized the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21) as the basis for estimating effort. Each team member independently reviewed the user stories and selected a value representing the expected complexity, risk, and amount of work required. When estimates differed significantly, the team discussed assumptions and reached a consensus before assigning a final story point value.

Agile Poker Estimation Results

Figure 7. Agile Poker Estimation Results

Planning Poker Results	
User Story	Story Points
User Authentication and Login	3
Create and Manage Projects	5
Create, Assign, and Track Tasks	5
Team Messaging and Collaboration	8
File Sharing Functionality	8
Project Dashboard and Reporting	5
Role-Based Access Control	5
AI Assistant Task Recommendations	13
AI Assistant Project Summaries	21
Total Estimated Effort: 73 Story Points	

Note. Planning Poker was used to estimate the relative effort required for major DoIT project features and user stories. Final estimates were determined through team discussion and consensus using the Fibonacci estimation scale.

Analysis

The Planning Poker session identified the AI-powered assistant features as the most complex components of the DoIT application. These features require additional logic, data

processing, and integration efforts compared to standard project management functionality. User authentication received the lowest estimate because it is a common feature with well-established implementation approaches. Messaging and file sharing received moderate estimates due to their integration requirements and potential security considerations.

The Planning Poker process helped the team establish a realistic understanding of project scope while encouraging collaboration and discussion regarding technical challenges. These estimates will be used to support sprint planning, task prioritization, and overall project scheduling throughout the remainder of the software development lifecycle.

Conclusion

The Agile Planning Poker session provided a structured approach for estimating development effort within the DoIT project. By involving all team members in the estimation process, the team developed a shared understanding of project requirements and complexity. The resulting story point estimates will help guide development priorities and support effective Agile project management practices throughout the DevOps lifecycle.

Meeting Notes

Date: 05/27/2026

Attendees:

- Angel Perez
- Daniel Nethala
- Justin Emerick
- Mathew Goncalves

Topics Discussed

Several topics were discussed throughout the meeting. For instance, the topology of the project was discussed, and several parts of the topology were divided among members of the group. Each group member will work on different parts of the topology to ensure full completion of the document. Another topic mentioned in the meeting was project scheduling. The Gantt chart for this project will be updated to reflect the current progress of the project development. Other topics include discussing the IBM report and how this report will be used as a template for our topology document, and the planning ‘poker’ estimation.

The planning ‘poker’ estimation session was explained throughout the meeting. Point estimates will be used to determine how long each task of the project will take to be completed. Furthermore, the team will collaborate again on Saturday to ensure that the document is completed and formatted properly to reflect industry standards. As a result, these are some of the topics discussed throughout the meeting.

Conclusion

This report provided an update on the development and planning of the DoIT application. Throughout this phase of the project, our team continued working on system models, planning, project scheduling, and Agile estimation. The diagrams and models created during this stage help provide a better understanding of how the application will function and how its components will interact with one another. The project plan and Gantt chart were updated to reflect the current progress of development. Team meetings and planning sessions also helped ensure that responsibilities were divided appropriately and that project goals remained on track. As the project continues, the team will focus on completing development tasks, testing application features, and preparing the system for deployment through the DevOps lifecycle.